Assignment – 18.2

Name: Yashaswini
Nagireddi
Batch:21
Ht.no:2303A51418

Task Description – 1 (Air Quality API Integration)
• Task:
Use AI assistance to develop a Python program that retrieves real-time air quality information for a specified city.
Instructions:
• Generate Python code using the requests library.
• Handle errors such as invalid city names, missing or incorrect API keys, and network timeouts.
• Extract and display AQI value, pollution level, and dominant pollutant in a readable format.
Sample Code:

```
import requests
city = "Delhi"
url = "https://api.example.com/airquality"
```

Expected Output:
• A Python script that successfully fetches and displays air quality data with proper error handling.

Prompt:
Generate Python code using the requests library to fetch real-time air quality data for a given city.
Include error handling for invalid city names, API key issues, and network failures.
Display AQI, pollution level, and dominant pollutant.

lab exam.py | 07-04-2026.py | 11-04-2026.py | BLACKBOX

lab 19 > 11-04-2026.py > ...

```python
import requests

def get_air_quality(city):
    API_KEY = "demo"   # use demo token (works for testing)

    url = f"https://api.waqi.info/feed/{city}/?token={API_KEY}"

    try:
        response = requests.get(url, timeout=5)
        data = response.json()

        if data["status"] != "ok":
            print("Invalid city or API issue")
            return

        aqi = data["data"]["aqi"]
        pollutant = data["data"]["dominentpol"]

        print("City:", city)
        print("AQI:", aqi)
        print("Dominant Pollutant:", pollutant)

    except requests.exceptions.RequestException:
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
City: Delhi
AQI: 95
Dominant Pollutant: pm25
PS C:\Users\dell\OneDrive\Desktop\AI>
```

powersh... | Python | Python | Python

Explanation:

This project retrieves **real-time air quality data** for a given city using an external API.

- The program sends an HTTP GET request to the API.
- The API responds with data in **JSON format**.
- From the response, important values like:
  - AQI (Air Quality Index)
  - Dominant pollutant

  are extracted and displayed.

Task Description – 2 (Language Translation API)

• Task:

Integrate a Language Translation API to translate text from English to two other languages.

Instructions:

• Use AI to generate API request code for translation.
• Validate user input text and target language codes.
• Handle API response errors and service unavailability.
• Display original and translated text clearly.

Sample Code:

```
import requests
text = "Welcome to AI Assisted Coding"
target_lang = "fr"
```

Expected Output:

• A working Python script that translates user input text with graceful error handling.

Prompt:

Generate Python code to translate English text into another language using an API.
Validate input and handle API errors. Display original and translated text.

```python
import requests

def translate_text(text, target_lang):
    # Input validation
    if not text.strip():
        print("Error: Text cannot be empty.")
        return

    valid_langs = ["fr", "es", "de", "hi"]
    if target_lang not in valid_langs:
        print("Error: Invalid language code.")
        return

    url = f"https://api.mymemory.translated.net/get?q={text}&langpair=e

    try:
        response = requests.get(url, timeout=5)
        response.raise_for_status()

        data = response.json()

        translated = data["responseData"]["translatedText"]
```

```
sktop/AI/lab 19/11-04-2026.py"
riginal: Welcome to AI Assisted Coding
ranslated: Bienvenue dans le codage assisté par IA
```

Explanation

This project translates English text into another language using a translation API.

- User provides input text and target language code.
- The program sends a request to the API.
- The API returns translated text.
- Both original and translated text are displayed.

Task Description – 3 (Public Book Information API)

• Task:

Connect to a public Books API to fetch information about a book using its title or ISBN.

Instructions:

• Prompt AI to write Python code for sending GET requests to the API.

• Handle errors such as book not found, invalid input, and rate limits.

• Display book title, author, publisher, and publication year.

Sample Code:

```
import requests
query = "Artificial Intelligence"
```

Expected Output:

• A Python program that retrieves and displays book details with robust error handling.

lab exam.py | 07-04-2026.py | 11-04-2026.py | BLACKBOX

lab 19 > 11-04-2026.py > ...

```python
#Generate Python code to fetch book details using a public Books
#Handle errors like book not found and invalid input. Display tit
import requests
def fetch_book_details(book_title):
    api_url = f"https://www.googleapis.com/books/v1/volumes?q=int
    try:
        response = requests.get(api_url)
        response.raise_for_status()  # Check for HTTP errors
        data = response.json()
        if 'items' in data and len(data['items']) > 0:
            book = data['items'][0]['volumeInfo']
            title = book.get('title', 'N/A')
            authors = ', '.join(book.get('authors', ['N/A']))
            publisher = book.get('publisher', 'N/A')
            published_date = book.get('publishedDate', 'N/A')
            print(f"Title: {title}")
            print(f"Author(s): {authors}")
            print(f"Publisher: {publisher}")
            print(f"Year: {published_date}")
        else:
            print("Book not found.")
    except requests.exceptions.RequestException as e:
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
Enter the book title: Artificial Intelligence: A Modern Approach
Title: Artificial Intelligence
Author(s): Stuart Jonathan Russell, Peter Norvig, Ernest Davis
Publisher: Prentice Hall
Year: 2010
```

powersh... | Python | Python | Python | powersh...

Prompt:

Generate Python code to fetch book details using a public Books API. Handle errors like book not found and invalid input. Display title, author, publisher, year.

Explanation

This project fetches book details using a public Books API.

- User enters a book title or keyword.
- The program sends a GET request to the API.
- It retrieves:
    - Title
    - Author

- Publisher
- Publication year

Task Description – 4 (Real-Time Application: Job Listings Aggregator)

• Scenario:

Develop a Job Listings Aggregator using a public Jobs API.

Requirements:

• Fetch the latest job postings for a given role or location.

• Handle errors such as invalid search terms, missing API keys, and request failures.

• Display job title, company name, location, and posting date in a numbered list.

• Implement a retry mechanism for failed API requests.

Sample Code:

```
import requests
keyword = "Python Developer"
```

Expected Output:

• A functional Python script that retrieves and displays job listings with proper error handling.

Prompt:

Generate Python code to fetch job listings using an API.
Display job title, company, location, and date.
Include retry mechanism and error handling.

9 > 11-04-2026.py > ...

```python
import requests
import time

def fetch_jobs(keyword, retries=3, delay=3):
    url = f"https://remotive.com/api/remote-jobs?search={keyword}"

    for attempt in range(retries):
        try:
            response = requests.get(url, timeout=5)
            response.raise_for_status()

            data = response.json()

            # DEBUG: show total jobs count
            print("Total jobs found:", len(data.get("jobs", [])))

            jobs = data.get("jobs", [])

            if not jobs:
                print("No jobs found. Try different keyword.")
                return

            print(f"\nTop Jobs for '{keyword}':\n")

            for i, job in enumerate(jobs[:5], 1):
                print(f"{i}. {job['title']}")
```

```
lab 19 > 11-04-2026.py > ...
1   import requests
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Total jobs found: 4

Top Jobs for 'python':

1. Fullstack Developer (US - ET)
   Company: Chooose
   Location: USA
   Date: 2026-04-08T12:36:43

2. Technical Support Operator
   Company: KnownHost
   Location: USA
   Date: 2026-03-27T09:51:22

3. Tech Lead Databricks Data Engineer
   Company: Mitre Media
   Location: USA, Canada, USA timezones
   Date: 2026-03-14T20:46:28

4. Full-Stack Developer
   Company: ELECTE S.R.L.
   Location: Worldwide
   Date: 2026-03-13T15:17:31

PS C:\Users\dell\OneDrive\Desktop\AI>
```

Explanation

This project collects job listings using a public Jobs API.

- **User provides a keyword (e.g., "python").**
- **The program fetches job data from API.**
- **Displays:**

- **Job title**
- **Company name**
- **Location**
- **Posting date**

Task Description – 5 Stock Market Data API Integration

• Task:

Use AI assistance to develop a Python program that retrieves real-time stock market data for a selected company.

Instructions:

• Generate Python code using the requests library to connect to a public Stock Market API.

• Validate user input for stock symbol.

• Handle common errors such as invalid symbols, API rate limits, missing API keys, and network failures.

• Extract and display stock name, current price, day high, day low, and percentage change.

Sample Code:

```
import requests
symbol = "AAPL"
```

• Expected Output:

A Python script that successfully fetches and displays stock market information with robust error handling.

Prompt:

Generate Python code to fetch real-time stock data using an API. Validate stock symbol and handle errors. Display price, high, low, and change.

lab exam.py | 07-04-2026.py | 11-04-2026.py | BLACKBOX

lab 19 > 11-04-2026.py > ...

```python
import requests

def get_stock(symbol):
    url = f"https://stooq.com/q/l/?s={symbol}&f=sd2t2ohlcv&h&e=csv"

    try:
        response = requests.get(url, timeout=5)
        response.raise_for_status()

        data = response.text.split("\n")[1].split(",")

        # ✅ Check invalid symbol
        if data[0] == "N/D":
            print("Invalid stock symbol.")
            return

        print("Stock:", symbol.upper())
        print("Date:", data[1])
        print("Time:", data[2])
        print("Open:", data[3])
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
Close Price: N/D
Stock: INVALID
Date: N/D
Time: N/D
Open: N/D
High: N/D
Low: N/D
Close Price: N/D
```

powersh... | Python | Python | Python | powersh... | Python

Explanation

This project retrieves real-time stock data for a company.

- User enters stock symbol (e.g., AAPL).
- The program fetches stock data from API.
- Displays:
  - Stock name
  - Current price
  - High & Low
  - Percentage change